



牛客竞赛

AC.NOWCODER.COM

2026 牛客 暑期多校训练营

子包命题组

HR.NOWCODER.COM



难度分布

- Easy: H, D, G
- Medium: I, F
- Medium-Hard: J, A
- Hard: C, L, E
- Mysterious: B, K

H - Hard Problem

题目大意

- 给定 n ，构造一个字典序最小的长度为 n 的排列 p ，使得相邻两项（ p_1 和 p_n 看作相邻）的绝对值都不是质数。

H - Hard Problem

题解

- 如果 $n - 1$ 不是质数，则 $1, 2, \dots, n$ 满足条件。
- 否则除了 $n = 3, 4, 6$ 无解外，可以构造出 $1, 2, \dots, n - 4, n, n - 1, n - 2, n - 3$ 。这是由于 $n - 1$ 是质数（一定是奇数）的情况下 $n - 4$ 一定是偶数（除了 2 以外都非质数）。简单分类讨论容易证明这是字典序最小的。

D - Divisibility

题目大意

- 给定无向图 G 和正整数 k , 求从 1 出发到达每个点, 且路径长度为 k 的倍数的最短路。

D - Divisibility

题解

- 核心观察是我们可以通过在一条边反复横跳来凑到 k 的倍数。由于来回一次不会改变路径长度的奇偶性，考虑分类讨论。
- 令 $d_{u,r}$ 为从 1 到 u ，路径距离模 2 为 r 的最短路。这可以由一次 bfs 简单求得。

D - Divisibility

题解

- 先考虑 k 是偶数的情况。
- 如果此时有 $d_{u,0} = \infty$ ，意味着不存在任何偶数长度的路径到达 u ，也就不可能是 k 的倍数了，直接输出 -1 。
- 否则答案为 $d_{u,0} + (-d_{u,0} \bmod k)$ 。

D - Divisibility

题解

- 再考虑 k 是奇数的情况。
- 如果此时有 $d_{u,0} = d_{u,1} = \infty$ ，意味着 u 不可达，直接输出 -1 。
- 否则，最终答案的路径长度可以为奇数或偶数，简单计算可以得到答案为 $\min(d_{u,1} + (-d_{u,1} \bmod k), d_{u,0} + (-d_{u,0} \bmod k) + k)$ 。



D - Divisibility

Bonus

- 如果变成有向图 $n \leq 10^3$, $m \leq n(n-1)$ 该怎么做?

G - Game Theory

题目大意

- 无向连通简单图，图中每个点度数不超过 3。图中存在一些特殊点。Alice 和 Bob 进行博弈。Alice 初始在某一个点上。两人轮流行动，Alice 先手。Alice 每轮可以选择一个相邻的点移动过去（不存在就输），Bob 每轮可以选择一条边删除。如果 Alice 移动到特殊点则胜。问有多少个初始点 Alice 必胜。

G - Game Theory

题解

- 首先对图做一个简单变换。对于每个特殊点 u ，设其邻居集合为 S_u ，则把这个点拆成 S_u 个特殊点，分别向 S_u 中的点连一条边。不难发现这不改变答案。
- 考虑如果 Bob 先手，有哪些点 Alice 必胜。
- 引理：令 W 为上述必胜点的集合，则有 $u \in W$ 当且仅当 u 是特殊点，或者 u 至少有两个邻居属于 W 。

G - Game Theory

题解

- 证明：由于图中的度数限制， W 的导出子图一定为一些二叉森林构成的集合。这说明如果 u 至少两个邻居属于 W ，则这两条分叉是完全独立的。如果 Bob 选择删掉某个分叉中的一条边，则 Alice 可以直接走另一个分叉，保证自己身处的节点永远属于 W 。
- 回到原题，则如果 Alice 先手，一个节点是必胜点当且仅当任何一个邻居属于 W 。
- W 可以通过拓扑排序简单得到。我们从所有特殊点开始，如果一个节点至少两个邻居属于 W 则把它插入队列中。

I - Integer Function

题目大意

- 对于非负整数 x ，定义 $f(x)$ 为 x 在二进制表示中 1 的个数。
- 给定 n 和 d ($0 \leq n, d < 2^{60}$)，求以下算式的值：

$$\left(\sum_{i=0}^n f(i) \cdot f(i+d) \right) \bmod 998244353$$

I - Integer Function

题解

- 考虑从高位到低位进行数位 DP。
- 但是加法的进位是从低位向高位传递的，与数位 DP 的顺序相反。
- 一个技巧是在状态中记录**当前位必须向高位提供的进位** c 。枚举当前位时，猜测低位传来的进位 nc 。若两者相同则合法。
- 状态设计： $f_{i,ish,c}$ ，表示处理到第 i 位，是否紧贴上界 n ，且要求向高位进位 c 。

I - Integer Function

题解

- DP 过程需维护 4 个信息：合法方案数 $\sum 1$ 、原数贡献 $\sum A$ 、加法后贡献 $\sum B$ 、以及乘积和 $\sum AB$ 。
- 考虑向后缀加入当前位（设原数当前位为 d ，加法后当前位为 r ），展开如下：

$$(A + d)(B + r) = AB + dB + rA + dr$$

- 根据上述展开式，可以利用低位传上来的 4 个维护值线性转移。总时间复杂度 $O(\log n)$ 。

F - Full Alphabet

题目大意

- 给定一个仅由小写字母组成的字符串 s ，求出有多少字母表 A ，满足 s 在 A 下是 Lyndon 串。

F - Full Alphabet

题解

- 首先，任意串的前缀都比他本身字典序小，所以如果 s 存在一个公共前后缀，则答案为 0。
- 由于 s 需要比它的所有后缀都小，考虑把 s 和其每个后缀做比较。令 $s[i:]$ 为 s 从第 i 个位置开始的后缀。

F - Full Alphabet

题解

- 考虑比较字典序的过程：找到第一个不一样的位置（最长公共前缀的下一个位置），看这个字符谁大谁小。于是我们可以找到 s 和 $s[i:]$ 的最长公共前缀，并令下一个位置上的字符分别为 c 和 c' 。这意味着要让 s 是 Lyndon 的，字母表 A 中 c 一定要在 c' 之前。求最长公共前缀这一步可以用 Z 函数线性解决。

F - Full Alphabet

题解

- 对每个 $2 \leq i \leq n$ 我们都可以得到一个上述的限制。把这些限制建模成一张有向图， $u \rightarrow v$ 当且仅当 u 必须在 v 之前，则任意一个合法的字母表其实都和这张图的一个拓扑排序唯一对应。问题转化为了有向图上的拓扑排序计数，状压 dp 即可。
- 时间复杂度 $O(n + \Sigma 2^\Sigma)$ ，其中 $\Sigma = 26$ 。

J - Just Add and Divide

题目大意

- 维护两个正数数列 a 和 b ，支持单点修改和区间查询 $\max_{l \leq i < j \leq r} \frac{a_i + b_i}{a_j + b_j}$ 。

J - Just Add and Divide

题解

- 先考虑静态单次询问。
- 引理：令 $k = \arg \max_{l \leq i \leq r} \frac{a_i}{b_i}$ ，则在最大的 $\frac{a_i + b_i}{a_j + b_j}$ 中一定有 $k \in \{i, j\}$ 。
- 证明：考虑任何一对 (i, j) ，且 $k \notin \{i, j\}$ 。不失一般性的令 $\frac{a_i}{b_i} \leq \frac{a_j}{b_j}$ 。则根据糖水不等式有：

$$\frac{a_i}{b_i} \leq \frac{a_i + a_j}{b_i + b_j} \leq \frac{a_j}{b_j} \leq \frac{a_j + a_k}{b_j + b_k} \leq \frac{a_k}{b_k}$$

J - Just Add and Divide

题解

- 在确定了 $i = k$ 后，问题变成了 $\max_{l \leq j \leq r \wedge j \neq k} \frac{a_k + a_j}{b_k + b_j}$ 。
- 注意到 $\frac{a_k + a_j}{b_k + b_j}$ 其实就是点 (b_k, a_k) 和 $(-b_j, -a_j)$ 之间直线的斜率。固定一个点在另一个点集里寻找斜率最大的点是一个经典结论，可以直接在点集的凸包上二分。

J - Just Add and Divide

题解

- 最后回到带修和区间查询。考虑把数组分成 $\sqrt{n}$ 大小的块。每次单点更新可以直接暴力重构块内凸包。每次区间查询可以 RMQ 先找到 k ，然后整块凸包二分，散块暴力的寻找最优的 j 。
- 默认 n 和 q 同阶的情况下时间复杂度 $O(n\sqrt{n}\log n)$ 。精细一点的优化和分析可以做到 $O(n\sqrt{n\log n})$ ，但在本题中不是必要的。

A - A Permutation Problem

题目大意

- 给定一个长度为 n 的排列 p 。
- 你可以进行任意次（包含零次）操作：选择一个位置 i ($2 \leq i \leq n-1$) 满足 $p_i > \max(p_{i-1}, p_{i+1})$ ，然后交换 p_{i-1} 和 p_{i+1} 。
- 求可以从 p 经过操作到达的不同排列数量，对 998244353 取模。

A - A Permutation Problem

题解

- 考虑操作的本质：如果一个数 x 很大，那么它左右两边比它小的数可以通过它进行交换。我们可以将 x 视作一座“桥”。
- 对于任何 $< x$ 的数，它们可以自由地跨过 x 进行交换。这意味着在不考虑其他限制的情况下，如果 $p_i = x$ ，位置 $i-1$ 和 $i+1$ 对于所有小于 x 的数是互相连通的。
- 在这样一个“可交换位置”的连通图内部，元素可以通过多次跨越桥梁实现任意的排列（类似于相邻交换操作能够生成整个对称群）。

A - A Permutation Problem

题解

- 为了不重复、不遗漏地统计，我们考虑将数字从大到小（从 n 到 1）依次确定最终位置。
- 当我们处理到数字 x （假设其初始在位置 j ）时，所有 $> x$ 的数字已经处理过，它们此时占据了若干位置，且相对顺序已经固定。
- 此时 x 可以在其当前所在连通块的任意**剩余可用位置**中选择。如果可用位置数为 S ，则 x 有 S 种选择。选择后，该连通块的剩余可用容量 S 减 1。

A - A Permutation Problem

题解

- 在确定完 x 的位置之后， x 本身也成为了一座合法的“桥”，并尝试将其左右两侧的位置 $j-1$ 和 $j+1$ 连通。
- 如果某侧连通块的 sz 为 0，意味着该连通块内的所有位置都已经被 $> x$ 的数字占满。
- 根据操作条件，桥 x 必须大于被交换的两个元素。既然该侧全是大于 x 的元素， x 就无法作为桥梁让元素通过。此时这条路是“堵死”的，因此绝对不能合并。

A - A Permutation Problem

题解

- 我们可以使用并查集来维护连通性和每个连通块的剩余可用大小 sz 。
- 初始时，每个位置各自为一个大小为 1 的连通块。
- 按从大到小的顺序遍历 x ，每次将答案乘上 x 初始位置 j 所在连通块的剩余大小 sz ，并将该大小减 1；随后（若 sz 合法）合并位置 $j-1$ 和 $j+1$ 的连通块。
- 整体时间复杂度 $O(n\alpha(n))$ 。

C - Cutting Trees

题目大意

- 给定一棵有根树。定义一个删除序列为：每次从树中选择一个还未删除的点，将其子树删掉。一个删除序列的分数为：每次删除之后计算树中剩余的节点数，对所有删除求和。求所有删除序列的分数之和。

C - Cutting Trees

题解

- 注意到一个删除序列的分数可以转化为每个节点的删除时刻之和减 n ，即如果一个节点在第 i 次操作中被删除（包括间接被删除），则其对分数的贡献为 $i - 1$ 。

C - Cutting Trees

题解

- 考虑树上 DP。定义以下 DP 状态： $f_{u,k}$ 代表 u 子树中有多少合法的删除序列（不需要把 u 也删掉）， $g_{u,k}$ 代表所有 $f_{u,k}$ 个序列执行之后， u 子树中剩余节点数之和， $h_{u,k}$ 代表代表所有 $f_{u,k}$ 个序列中，被删除的节点的删除时刻之和。
- 对于空子树，有 $f = 1$ ， $g = h = 0$ 。

C - Cutting Trees

题解

- 考虑转移。对于两个不交的子树 v 和 w ，设第一个子树有一个长度 i 的删除序列，第二个子树有一个长度 j 的删除序列。注意 i 的删除序列和 j 的是完全独立的，交错方案数为 $\binom{i+j}{i}$ ，所以有

$$f_{u,i+j} \leftarrow f_{u,i+j} + \binom{i+j}{i} f_{v,i} f_{w,j}。$$

- 由于交错顺序并不会影响最终剩余节点数，所以也有

$$g_{u,i+j} \leftarrow g_{u,i+j} + \binom{i+j}{i} (g_{v,i} f_{w,j} + g_{w,j} f_{v,i})。$$

C - Cutting Trees

题解

- 对于 h ，考虑第一个序列中的第 r 次操作。在所有交错方案中，这次操作前面平均有 $\frac{rj}{i+1}$ 次第二个序列的操作，所以它的平均位置为 $r + \frac{rj}{i+1}$ 。再乘交错方案数可以得到第 r 次操作在所有合并得到的序列中的位置之和为 $\binom{i+j}{i} \frac{rj}{i+1} = r \binom{i+j+1}{i+1}$ 。所以有

$$h_{u,i+j} \leftarrow h_{u,i+j} + \binom{i+j+1}{i+1} h_{v,i} g_{w,j} + \binom{i+j+1}{j+1} h_{w,j} g_{v,i}。$$
- 最后把当前子树根的影响加进去。如果不选 u 进入序列，则剩余节点数增加 1，别的不变；否则 u 一定在序列末尾，且删除剩余的所有节点。
- 时间复杂度 $O(n^2)$ 。

L - Longest Palindromic Substring Queries

题目大意

- 给定一个字符串 s 。回答 q 次询问，每次需要计算 s 删掉一个连续子串并将两部分拼起来后的最长回文子串长度。

L - Longest Palindromic Substring Queries

题解

- 答案可以拆分为三部分：只包含在前缀里的回文串，只包含在后缀里的回文串，和横跨前后缀的回文串。前面两部分可以简单预处理得到。主要考虑第三部分。
- 令 A 为前缀， B 为后缀。横跨 A 和 B 的回文串一定是 $\text{rev}(V) + M + V$ 的形式，其中 M 为 A 的某个回文后缀。也就是说我们对 A 的每个回文后缀 M ，找到 $\text{rev}(A)[|M|:]$ 和 B 的最长公共前缀 V ，用 $|M| + 2|V|$ 更新答案。

L - Longest Palindromic Substring Queries

题解

- 难点在于 A 最坏会有 $O(n)$ 个回文后缀，暴力更新的复杂度是不对的。
- 我们可以用到回文后缀的性质：一个字符串的所有回文后缀可以被划分成 $O(\log n)$ 个等差数列的形式，且任意一个等差数列一定满足 $W, D + W, D^2 + W, \dots, D^k + W$ 的形式，其中 W, D 为字符串。

L - Longest Palindromic Substring Queries

题解

- 这里不加证明的给出一个结论：在一个等差数列里， $\text{rev}(A)[|M|:]$ 和 B 的最长公共前缀长度是单峰的。具体来说，这个位置只会在 $\frac{\text{lcp}(D^\infty, \text{rev}(B)) - \text{lcp}(D^\infty, \text{rev}(A)[|D^k + W|])}{|D|}$ 附近的常数个点取到。
- 所以每次询问的复杂度变成了 $O(\log n)$ 次求最长公共前缀。等差数列可以通过回文树求出，而最长公共前缀可以通过后缀数组求出。最终复杂度 $O(n \log n + q \log n)$ 。



B - CF2071B

题目大意

- 给定 n ，构造一个长度为 n 的排列，最小化总和为完全平方数的区间数。

B - CF2071B

题解

- 首先考虑哪些区间和必须是完全平方数。
- 对于 $1^2, 2^2, 3^2, \dots, |\sqrt{n}|^2$ ，它们都必然会作为长度为 1 的区间出现在序列中。对于整个序列和 $1 + 2 + \dots + n$ ，若它是完全平方数，我们也不能避免它出现，因此 $M(n) \geq |\sqrt{n}| + [n \in \{8, 49, 288, 1681\}]$ 。事实上，这个不等式对 $1 \leq n \leq 5000$ 都取到等号。

B - CF2071B

题解

- 因此目标变为：构造一个排列，使得所有长度不为 1 也不为 n 的区间和都不是完全平方数。
- 考虑暴力怎么做。暴力搜索做法是依次确定 $a_1, a_2, \dots, a_n$ ，当填入 a_i 时，检查 $\sum_{k=j}^i a_k$ ($1 \leq j < i$) 和 $\sum_{k=i+1}^n a_k = \frac{n(n+1)}{2} - \sum_{k=1}^i a_k$ ：它们都不能是完全平方数。随后继续搜索 a_{i+1} ，若搜索不到就回溯。
- 事实上，这个做法对 $n = 5000$ 可以秒算出一组解。

B - CF2071B

题解

- 然而，对于剩下的不足 10% 的情形，进行更多的搜索无助于解决问题。这是因为每当 a_i 被填入时，隐式规定了 $\sum_{j=i+1}^n a_j - a_n$ 不能是完全平方数，在接近结尾时总共禁止了 $O(n)$ 个，因此有一定概率不存在任何合法的 a_n ，这种情况下无论如何搜索都不能在合理时间内得到答案。
- 考虑在搜索超过 $2n$ 个状态后就从头开始重新搜索，可以通过此题。经过对每个 n 的 5000 轮验证，以随机的顺序枚举下一个元素进行搜索，对于任意的 $1 \leq n \leq 5000$ 都有 90% 的概率在 $2n$ 个状态内得到答案。

K - Konstruct

题目大意

- 将纸带每连续 16 位看成一个整数块，记为 $b_i = c_{16i \dots 16i+15}$ 。
- 初始纸带上的整数依次为 $b_0 = 0, \quad b_1 = n, \quad b_2 = k, \quad b_{i+3} = a_i$ 。
- 需要构造图灵机，使得停机时 $b_0 = |i \mid a_i > k|$ 。

K - Konstruct

基本操作：减一并判断零

- std 中的

check zero *if*

同时完成了判断一个 16 位整数是否为零，以及将其减一。

- 从最低位向最高位扫描。状态中的 *f* 表示是否已经遇到过第一个 1。
- 在还没有遇到 1 时：

$$0 \longrightarrow 1, \quad 1 \longrightarrow 0,$$

并在读到 1 后令 $f = 1$ 。

- 遇到第一个 1 后，剩余高位保持不变。这正是二进制减一：

K - Konstruct

$$x \longrightarrow x - 1.$$

K - Konstruct

基本操作：四个块的循环移动

- std 用大量展开的 `swap left`, `swap right` 状态，实现四个相邻 16 位整数块的循环移动。
- 正向移动执行 $(x, y, z, w) \longrightarrow (w, x, y, z).$ 。
- 反向移动执行 $(w, x, y, z) \longrightarrow (x, y, z, w).$ 。
- 在实际使用中，计数器已经先执行了减一。因此正向阶段的一轮操作为 $(x, y, z, w) \longrightarrow (w, x - 1, y, z).$ 。
- 反向阶段的一轮操作为 $(w, x, y, z) \longrightarrow (x, y - 1, z, w).$ 。
- 这样可以让需要使用的几个变量沿着纸带移动，而不必每处理一个元素都返回纸带开头。

K - Konstruct

初始化

- 初始指针位置为 $p_0 = 16$, 即 n 的最低位。
- `init()` 将 b_1 中的 n 按位复制到 b_0 , 但不清除原来的 n 。
- 初始化结束后, 纸带上的整数为 $(n, n, k, a_0, a_1, \dots, a_{n-1})$ 。
- 指针回到 b_0 的最低位, 进入正向阶段。
- 这里复制出的第一个 n 将作为正向扫描的循环计数器, 原来的 n 会被保留下来, 供反向扫描使用。

K - Konstruct

正向阶段

- 正向阶段的目标是将 n 和 k 搬到整个数组右侧，并在数组与 n 之间留下一个零块作为初始答案。
- 在处理完前 i 个元素时，纸带满足
$$(a_0, a_1, \dots, a_{i-1}, n - i, n, k, a_i, a_{i+1}, \dots, a_{n-1}).$$
- 指针位于计数器 $n - i$ 的最低位。
- 初始时 $i = 0$ ，纸带为 $(n, n, k, a_0, a_1, \dots, a_{n-1})$ ，所以不变量成立。

K - Konstruct

正向阶段的一轮操作

- 如果当前计数器 $n - i$ 非零，首先将它减一，然后对接下来的四个块执行循环右移： $(n - i, n, k, a_i) \longrightarrow (a_i, n - i - 1, n, k)$.
- 因此整条纸带变成 $(a_0, \dots, a_i, n - i - 1, n, k, a_{i+1}, \dots, a_{n-1})$.
- 这恰好是 i 增加一后的不变量。
- 指针也随窗口向右移动一个整数块，所以每轮只需要在常数长度的区域内移动。

K - Konstruct

正向阶段结束

- 执行 n 轮后，计数器变为零，纸带为 $(a_0, a_1, \dots, a_{n-1}, 0, n, k)$.
- 此时正向阶段的零判断发现当前块为零。
- std 将临时写入的全 1 清回全 0，然后进入反向阶段。
- 中间的零块将作为答案的初始值： $ans = 0$.
- 现在 n 、 k 和答案都位于数组右侧，可以开始从右向左处理数组。

K - Konstruct

反向阶段不变量

- 定义 $s_i = |j| \mid i \leq j < n, a_j > k|$.
- 当还剩下 $a_0, \dots, a_{i-1}$ 未处理时, 纸带满足 $(a_0, \dots, a_{i-1}, s_i, i, k, a_i, \dots, a_{n-1})$.
- 其中:
 - s_i 是已经处理的后缀中的答案;
 - i 是剩余未处理元素数;
 - k 位于计数器右侧;
 - 已处理的元素被依次移动到 k 的右侧。
- 反向阶段开始时 $i = n$, 并且 $s_n = 0$, 所以不变量成立。

K - Konstruct

反向阶段的一轮操作

- 当 $i > 0$ 时，当前需要处理的元素是 a_{i-1} 。
- 首先将计数器 i 减一，然后对四个块 $(a_{i-1}, s_i, i-1, k)$ 执行循环左移，得到 $(s_i, i-1, k, a_{i-1})$ 。
- 因此纸带变成 $(a_0, \dots, a_{i-2}, s_i, i-1, k, a_{i-1}, a_i, \dots, a_{n-1})$ 。
- 此时答案、计数器、 k 都向左移动了一个块，而当前元素 a_{i-1} 被移动到了 k 的右侧。

K - Konstruct

比较 (a_{i-1}) (k)

- std 从最高位到最低位比较 k 和 a_{i-1} 。
- 状态

backward compare start j

读取 k 的第 j 位，并将这一位保存在状态中。

- 指针向右移动 16 格，读取 a_{i-1} 的第 j 位。
- 如果 $k_j < a_{i-1,j}$ ，则立即得到 $a_{i-1} > k$ 。
- 如果 $k_j > a_{i-1,j}$ ，则立即得到 $a_{i-1} < k$ 。
- 如果两位相等，则继续比较下一低位。最低位也相等时，两个数相等，
不计入答案。

K - Konstruct

答案加一

- 如果比较结果为 $a_{i-1} > k$, 则对答案块 s_i 执行一次二进制加一。
- 从最低位开始维护进位:
 - $0 \rightarrow 1$ 后进位结束;
 - $1 \rightarrow 0$ 并继续向高位进位。
- 进位结束后, 剩余高位直接保持不变。
- 因而新的答案为 $s_{i-1} = s_i + [a_{i-1} > k]$ 。
- 一轮结束后得到 $(a_0, \dots, a_{i-2}, s_{i-1}, i-1, k, a_{i-1}, \dots, a_{n-1})$, 所以反向阶段不变量继续成立。

K - Konstruct

停机

- 当 $i = 0$ 时，纸带形如 $(s_0, 0, k, a_0, a_1, \dots, a_{n-1})$.
- 此时反向阶段的零判断发现计数器为零，直接进入停机状态。
- 根据定义， $s_0 = |j| \mid 0 \leq j < n, a_j > k|$.
- s_0 恰好位于第零个整数块，即纸带单元格 0 到 15。
- 因此停机时答案位于题目要求的位置。

K - Konstruct

状态数

- std 将所有固定长度的移动和所有 16 位循环直接展开成有限状态。
- 状态生成器使用字符串描述状态，并通过

`id(string)`

为每种状态分配编号。

- 各阶段结束时的状态数为：

阶段	当前总状态数
init	817
forward	4993

K - Konstruct

K - Konstruct

步数分析

- 初始化固定执行 559 步。
- 正向阶段每处理一个元素固定执行 2127 步。
- 正向阶段结束并进入反向阶段至多再执行 49 步。
- 反向阶段每处理一个元素至多执行 2720 步。最坏情况发生在比较需要一直进行到最低位时。
- 最后的零判断需要 17 步。因此总步数不超过
$$559 + 2127n + 49 + 2720n + 17 = 4847n + 625.$$
- 因为 $n < 2^{16}$, $4847n + 625 \leq 4847 \cdot 65535 + 625 = 317648770 < 4 \cdot 10^8$.
- 所以执行步数满足限制。

THANKS!

AC.NOWCODER.COM